



## Week 8 Lecture 4

|             |                            |
|-------------|----------------------------|
| ▼ Class     | BSCCS2003                  |
| 🕒 Created   | @October 24, 2021 11:38 AM |
| 🔗 Materials |                            |
| # Module #  | 47                         |
| ▼ Type      | Lecture                    |
| # Week #    | 8                          |

## Client side computations and Security Implications

### Validation

- Server-side validation essential
  - No guarantees that request actually came from a given front-end
- But some client-side validation can reduce hits on the server
- Example → E-mail, date range, sanitization (no invalid characters) etc.
- Similar validation to backend, but now in front-end script
- Extra work, but better user experience

[https://developer.mozilla.org/en-US/docs/Learn/Forms/Form\\_validation](https://developer.mozilla.org/en-US/docs/Learn/Forms/Form_validation)

### Inbuilt HTML5 form controls

- Partial validation added by HTML5 standard
- `required` → mandatory field
- `minlength, maxlength` → for text fields
- `min, max` → for numeric values
- `type` → for some specific predefined types
- `pattern` → regular expression pattern match

**Important** → older browsers may not all these features

Is backward compatibility essential for your app?

## JavaScript validation

Constraint Validation API

[https://developer.mozilla.org/en-US/docs/Web/API/Constraint\\_validation](https://developer.mozilla.org/en-US/docs/Web/API/Constraint_validation)

- Supported by most browsers
- Much more complex validation possible

**Remember** → not a substitute for server-side validation

```
<form>
  <label for="mail">I would like you to provide me with an e-mail address:</label>
  <input type="email" id="mail" name="mail">
  <button>Submit</button>
</form>
```

```
const email = document.getElementById("mail");

email.addEventListener("input", function (event) {
  if (email.validity.typeMismatch) {
    email.setCustomValidity("I am expecting an e-mail address");
  } else {
    email.setCustomValidity("");
  }
});
```

## Captcha

- **Problem** → scripts that try to automate web-pages
- Can generate large number of reports in short time - server load
- Railway Tatkal, CoWin appointments, etc

### Solution

- Prove that you are a human



- Limited number of clicks per possible unit time
- Script on the page will generate some token - server will reject requests without that token

## Crypto-mining?

- JavaScript is a "complete" language
- Can implement any computation with JavaScript
- Modern JS engines are very powerful and fast
  - Can even access system graphics processor (GPU) for rendering, etc.
- Run a simple page that loads and runs a JS script

- Script will send results back to the server through async calls
- Client may not even be aware

## Security Implications

### Sandboxing

- Should JS be run automatically on every page
  - Yes → provides significant capabilities
  - No → what if the page tries to load local files and send them to the server?
- Sandbox → secure area that JS engine has access to
- Cannot access files, network resources, local storage
- Similar to Virtual Machines, but at a higher level (JS interpreter)

### Overload and DoS

- DoS → Denial of Service
- Run a script that takes over the browser engine and runs at high load
- Difficult to even navigate away from the page, or close the page
- Potential exploit bugs in the browser
- Server attack:
  - Replace some popular JS file with a bad version
  - Will be loaded by a large number of sites, users → can write script to access some other site
  - Target site will be hit by a large number of requests from several sources, very difficult to control

### Access to native resources

- Can JS be used to write fully native applications?
- Access to resources like local storage, sensors (tilt, magnetometer, camera)
- Can be permitted explicitly through the browser

Can also be compiled directly to native resources

- Reduce browser overheads
- Smoother interaction with the system

### Summary

- Frontend experience determined by browser capabilities
  - Basic HTML + CSS rendering → styling
  - JavaScript/client-side scripting for user interaction, smoother integration
- Native clients possible
- Potentially serious security implications
- Always validate data again at the server, do not assume client validation (Never trust the client)
  - HTTP is stateless → server cannot assume client was in a particular state